Learning Predictive World Models with Curiosity-Driven Exploration

Aditya Randive

Northeastern University, Master of Science in Artificial Intelligence

April 26, 2026

Abstract

This paper presents a from-scratch implementation combining the World Models framework (Ha & Schmidhuber, 2018) with the Intrinsic Curiosity Module (Pathak et al., 2017) for the LunarLander-v3 environment. The system learns a compressed latent representation of the environment through a Variational Autoencoder (VAE), predicts future dynamics using a Mixture Density Recurrent Neural Network (MDRNN), generates curiosity-driven intrinsic rewards through an ICM, and trains a controller policy using Proximal Policy Optimization (PPO). All core components—the VAE, custom LSTM, Mixture Density Network, and ICM—were implemented from scratch in PyTorch without relying on pre-built modules. The project demonstrates the complete World Models architecture on a controlled testbed, achieving positive training rewards (+145) and documenting the challenges of model-based reinforcement learning including distribution shift, posterior collapse, and the critical role of controller optimization methodology. The project combines ideas from three major papers in the field and includes over 500,000 parameters across four custom neural network architectures.

1 Introduction

Reinforcement learning (RL) has achieved remarkable success in game-playing and robotics, yet two fundamental challenges persist: high sample complexity and sparse rewards. Model-free methods such as DQN and policy gradient algorithms learn purely from trial and error, requiring millions of environment interactions before useful policies emerge. When feedback is rare or delayed, standard exploration methods such as ϵ-greedy struggle to discover rewarding behaviors, as the probability of randomly stumbling upon a useful trajectory decreases exponentially with its length.

These limitations motivate two research directions that this project combines: predictive world models and curiosity-driven exploration.

1.1 World Models (Ha & Schmidhuber, 2018)

Ha and Schmidhuber (2018) introduced a framework where an agent learns an internal model of its environment and uses that model to make decisions. The architecture consists of three components, each inspired by a different aspect of human cognition:

The **Vision model (V)** uses a Variational Autoencoder (VAE) to compress high-dimensional observations into a compact latent vector z. Just as humans do not process every pixel of their visual field, the VAE learns to extract the essential features of each observation while discarding noise. The VAE is trained to reconstruct its inputs through a bottleneck, learning a compressed representation that preserves the information needed for downstream tasks.

The **Memory model (M)** uses a recurrent neural network—specifically, a Mixture Density RNN (MDRNN)—to predict the probability distribution over the next latent state given the current state and action: $P(z_{t+1}|z_t, a_t)$. This gives the agent the ability to imagine what will happen next without interacting with the real environment. The use of a mixture of Gaussians rather than a single Gaussian allows the model to capture multimodal dynamics, where the same state-action pair could lead to multiple plausible outcomes.

The **Controller (C)** is a deliberately simple policy—in the original paper, a single linear layer—that maps the current latent state and memory to actions. The simplicity of the controller is intentional: because V and M handle perception and prediction, C only needs to learn the mapping from understood states to optimal actions. Ha and Schmidhuber trained C using CMA-ES (Covariance Matrix Adaptation Evolution Strategy), a gradient-free evolutionary optimization method.

A key capability of this architecture is **dream training**: because M can predict future states, the controller can be trained entirely inside the model's imagination, without any real environment interaction. This is particularly valuable in domains where real interaction is expensive, dangerous, or slow—such as robotics, autonomous driving, or medical applications.

The World Models paper demonstrated this approach on two environments: a car racing task (CarRacing-v0) and a VizDoom environment. On CarRacing, the agent learned to drive by first learning a visual model of the track, then practicing entirely in its dreams. The paper inspired a lineage of increasingly powerful model-based RL systems, including DreamerV1 (Hafner et al., 2020), DreamerV2, and DreamerV3 (Hafner et al., 2023), the latter of which solved the diamond collection task in Minecraft using the same fundamental

V-M-C architecture.

1.2 Curiosity-Driven Exploration (Pathak et al., 2017)

Pathak et al. (2017) proposed the Intrinsic Curiosity Module (ICM), which addresses the exploration problem by generating reward signals from the agent's own prediction errors. The core insight is that prediction error in a learned feature space serves as a natural measure of novelty: states that the agent cannot yet predict are worth exploring.

The ICM consists of three components. A **feature encoder** ϕ maps observations into a learned feature space. A **forward model** predicts the next state's features given the current features and action: $\hat{\phi}_{t+1} = f(\phi(s_t), a_t)$. The prediction error $\|\hat{\phi}_{t+1} - \phi(s_{t+1})\|^2$ serves as the intrinsic reward. An **inverse model** predicts the action taken between two consecutive states: $\hat{a}_t = g(\phi(s_t), \phi(s_{t+1}))$. The inverse model is crucial because it trains the feature encoder to focus on aspects of the environment that the agent can actually influence, solving the "noisy TV problem"—where uncontrollable stochastic elements (like a television displaying random static) would produce permanently high prediction error and trap the agent in unproductive exploration.

Pathak et al. demonstrated that ICM-augmented agents could learn to play VizDoom and Super Mario Bros. with no extrinsic reward at all—the curiosity signal alone was sufficient to drive meaningful exploration and skill acquisition.

1.3 This Project: Combining Both Approaches

This project combines World Models with ICM, implementing the complete system from scratch on the LunarLander-v3 environment. The combination is theoretically motivated: the VAE's latent space provides a filtered, compressed representation that makes the curiosity signal less noisy than operating on raw observations. The MDRNN's hidden state provides temporal context that enriches the controller's input. The ICM provides dense reward at every timestep, addressing the sparse reward challenge that model-based methods alone cannot solve.

LunarLander-v3 was selected as a testbed because it features physics-based dynamics where small actions have compounding consequences, making it suitable for evaluating both the world model's prediction accuracy and the curiosity module's exploration effectiveness. The environment's manageable state space (8 dimensions, 4 discrete actions) allows thorough experimentation with multiple optimization approaches and hyperparameter configurations.

The project further explores three different controller optimization strategies: CMA-ES (as in the original

paper), dream training (training inside the MDRNN's imagination), and PPO (a modern policy gradient method), providing a comparative analysis of these approaches within the World Models framework.

2 Methods

2.1 Environment

All experiments were conducted using the LunarLander-v3 environment from the Gymnasium library (Towers et al., 2024). The environment provides an 8-dimensional continuous state vector consisting of horizontal position (x), vertical position (y), horizontal velocity (v_x), vertical velocity (v_y), angle (θ), angular velocity (ω), and two boolean leg contact indicators. The action space consists of four discrete actions: do nothing, fire left engine, fire main engine, and fire right engine. The reward structure includes $+100$ to $+140$ for landing between the flags, -100 for crashing, $+10$ for each leg contact, and small penalties for engine usage (-0.3 for main engine, -0.03 for side engines per frame). An episode terminates when the lander crashes, lands, or after 1000 timesteps.

2.2 Vision Model: Variational Autoencoder (108,488 parameters)

The VAE compresses the 8-dimensional observation into a 32-dimensional latent space. Although the latent dimension exceeds the input dimension, this expansion enables the VAE to learn a richer, disentangled representation where nonlinear combinations of raw state variables (such as "falling fast at a steep angle") are represented as distinct directions in latent space. The encoder consists of three fully connected layers ($8 \to 64 \to 128 \to 256$) with ReLU activations, splitting into two parallel heads for the mean (μ) and log-variance ($\log \sigma^2$) of the approximate posterior. The decoder mirrors the encoder architecture ($32 \to 256 \to 128 \to 64 \to 8$) with no activation on the output layer.

The VAE loss combines mean squared error reconstruction loss with KL divergence regularization:

$$\mathcal{L}_{VAE} = \mathcal{L}_{\text{recon}} + \beta \cdot D_{KL}(q(z|x) \| \mathcal{N}(0, I)) \tag{1}$$

The KL divergence has a closed-form solution for Gaussian distributions:

$$D_{KL} = -\frac{1}{2} \sum_{j=1}^{d} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \tag{2}$$

To prevent posterior collapse—a known issue with large latent dimensions where the encoder ignores input and outputs the prior—we employ β-warmup, linearly increasing β from 0 to 0.5 over the first 100

epochs. Capping β at 0.5 rather than 1.0 was found empirically to prevent collapse while maintaining sufficient regularization. The reparameterization trick ($z = \mu + \sigma \cdot \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$) enables gradient flow through the sampling operation.

2.3 Memory Model: MDRNN (383,557 parameters)

The Memory model combines a custom LSTM cell (300,032 parameters) with a Mixture Density Network head (83,525 parameters).

Custom LSTM. The LSTM was implemented from scratch with four gates (forget, input, candidate, output), each consisting of a separate linear transformation. The input at each timestep is the concatenation of the current latent state z_t (32D) and the one-hot encoded action a_t (4D), totaling 36 dimensions. The hidden state dimension is 256. Each gate computes a linear transformation of the concatenated input and previous hidden state ($292D \to 256D$), with sigmoid activations on the forget (f_t), input (i_t), and output (o_t) gates, and tanh on the candidate gate (g_t). The cell state update follows:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \tag{3}$$

$$h_t = o_t \odot \tanh(c_t) \tag{4}$$

The cell state acts as a highway that carries information across many timesteps, with the forget gate controlling what to discard and the input gate controlling what new information to add. This architecture addresses the vanishing gradient problem that prevents standard RNNs from learning long-range dependencies.

Mixture Density Network. The MDN head converts the LSTM hidden state into a mixture of five Gaussian distributions over the 32-dimensional latent space. Three parallel linear layers produce:

- Mixture weights π: Linear(256,5) with softmax (probabilities sum to 1)
- Means μ: Linear(256,160) reshaped to 5×32
- Standard deviations σ: Linear(256,160) with softplus activation, reshaped to 5×32

Softplus ($\log(1 + e^x)$) was chosen over exponential activation for numerical stability—softplus grows linearly for large inputs while exp grows exponentially, preventing gradient explosion during training.

The MDRNN loss is the negative log-likelihood under the mixture model:

$$\mathcal{L}_{MDRNN} = -\log \sum_{k=1}^{K} \pi_k \cdot \mathcal{N}(z_{t+1} | \mu_k, \sigma_k^2) \tag{5}$$

All computations are performed in log space using the logsumexp trick for numerical stability, preventing underflow when multiplying 32 small Gaussian probability values. Gradient clipping at norm 1.0 prevents gradient explosion during backpropagation through long sequences.

2.4 Intrinsic Curiosity Module (13,060 parameters)

The ICM operates on the VAE's latent space rather than raw observations, leveraging the already-filtered representation. It consists of three sub-networks:

The **Feature Encoder** (Linear(32,64) + ReLU $\rightarrow$ Linear(64,32)) transforms latent states into action-relevant features $\phi(z)$. This network is shared between the forward and inverse models and is trained primarily through the inverse model's gradients.

The **Forward Model** (Linear(36,64) + ReLU $\rightarrow$ Linear(64,32)) takes concatenated current features and one-hot action as input and predicts the next state's features. The prediction error $\|\hat{\phi}_{t+1} - \phi(z_{t+1})\|^2$ serves directly as the intrinsic reward.

The **Inverse Model** (Linear(64,64) + ReLU $\rightarrow$ Linear(64,4)) takes concatenated features of consecutive states and predicts the action taken between them, trained with cross-entropy loss.

A critical implementation detail is the use of `.detach()` on the target features $\phi(z_{t+1})$ in the forward loss computation. Without detachment, the forward model's gradients would flow back through the feature encoder via the target, allowing the forward model to trivially reduce its prediction error by collapsing the feature representation to a constant—rather than learning to predict accurately. The combined ICM loss weights the inverse loss at 80% and the forward loss at 20%, following Pathak et al. (2017):

$$\mathcal{L}_{ICM} = 0.8 \cdot \mathcal{L}_{\text{inverse}} + 0.2 \cdot \mathcal{L}_{\text{forward}} \tag{6}$$

The total reward at each timestep is:

$$r_{\text{total}} = r_{\text{extrinsic}} + \eta \cdot r_{\text{intrinsic}} \tag{7}$$

where $\eta = 0.0001$ scales the curiosity reward relative to the environment reward.

2.5 Controller Optimization

Three controller optimization strategies were explored:

CMA-ES (original approach). Following Ha and Schmidhuber (2018), the controller was initially a single linear layer ($288D \rightarrow 4D$, 1,156 parameters) trained with CMA-ES (Hansen, 2016). CMA-ES

maintains a multivariate Gaussian distribution over the parameter space and iteratively refines its mean, covariance matrix, and step size based on fitness evaluations. Each candidate controller is evaluated by running complete episodes, and the top-performing candidates inform the next generation's distribution.

Dream training. The controller was evaluated inside the MDRNN's imagination rather than the real environment. Starting from a real initial state, the MDRNN predicts future states and a learned reward predictor (MLP: $32 \to 64 \to 32 \to 1$) estimates rewards from latent states. This enables thousands of evaluations per minute since no real environment interaction is needed.

PPO (final approach). Proximal Policy Optimization (Schulman et al., 2017) was integrated via the stable-baselines3 library (Raffin et al., 2021). PPO learns from per-timestep advantage estimates using the clipped surrogate objective:

$$\mathcal{L}^{CLIP} = \min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right) \tag{8}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio and $\hat{A}_t$ is the estimated advantage. The clipping with $\epsilon = 0.2$ prevents destructively large policy updates, ensuring stable training.

The PPO controller was integrated through a custom Gymnasium environment wrapper (`WorldModelEnv`) that internally manages the VAE encoding, MDRNN hidden state updates, and ICM intrinsic reward computation, presenting PPO with a 288-dimensional observation space $[z_t, h_t]$ and the augmented reward signal $r_{\text{extrinsic}} + \eta \cdot r_{\text{intrinsic}}$.

2.6 Training Pipeline

The training pipeline proceeds in six sequential phases:

1. **Data collection:** Collect 200 episodes using a random policy from LunarLander-v3, yielding approximately 20,000 transition tuples.

2. **VAE training:** Train for 200 epochs with Adam optimizer (lr=0.001), mini-batch size 128, and β-warmup from 0 to 0.5 over the first 100 epochs.

3. **Latent encoding:** Encode all observations into latent sequences using the trained encoder's mean μ (deterministic, no sampling).

4. **MDRNN training:** Train for 150 epochs on the latent sequences with gradient clipping at norm 1.0.

5. **ICM training:** Train for 100 epochs on transition tuples (z_t, a_t, z_{t+1}) with mini-batch size 128.

6. **Controller training:** Train the PPO controller for 200,000 timesteps (approximately 1 minute on CPU) through the `WorldModelEnv` wrapper.

All neural networks were implemented in **PyTorch** (Paszke et al., 2019). The PPO implementation uses the **stable-baselines3** library (Raffin et al., 2021). The environment is provided by **Gymnasium** (Towers et al., 2024). All source code is available at

2.7 Evaluation Methodology

The system was evaluated at multiple levels to provide systematic assessment:

Component-level metrics: VAE quality was assessed through reconstruction loss and KL divergence. MDRNN quality was measured by negative log-likelihood on held-out transitions. ICM quality was evaluated through inverse model accuracy (action prediction) and forward model prediction error.

Training metrics: PPO's built-in logging tracked mean episode reward, episode length, entropy loss, explained variance, and policy gradient loss throughout training.

Evaluation metrics: Final performance was measured as mean episode reward over 10 episodes in the real LunarLander-v3 environment with $\eta = 0$ (no intrinsic reward), isolating pure extrinsic task performance.

Comparative analysis: CMA-ES and PPO were compared under identical world model representations to assess the impact of the optimization method. Multiple η values (0.01, 0.001, 0.0001, 0.0) were tested to quantify ICM's contribution.

3 Results

3.1 Component Training Results

Table 1 summarizes the training metrics for each component, demonstrating that all parts of the system learned meaningful representations.

Component	Start	End	Interpretation
VAE recon loss	6.81	0.66	Good reconstruction
VAE KL divergence	0.07	0.92	Active latent space, no collapse
MDRNN NLL	-95	-153	Accurate dynamics prediction
ICM inverse loss	1.39	0.32	75% action prediction (vs 25% random)
ICM forward loss	0.07	2.96	Forward error increased as features specialized

Table 1: Training metrics for individual components across all training epochs.

The VAE's KL divergence rising to 0.92 (with $\beta = 0.5$) indicates the latent space was being actively utilized—each dimension encodes meaningful variation rather than collapsing to the prior. The MDRNN's

decreasing negative log-likelihood demonstrates successful learning of environment dynamics in latent space. The ICM's inverse loss dropping from 1.39 ($\ln(4) \approx 1.39$, equivalent to random guessing on 4 classes) to 0.32 indicates the feature encoder learned action-relevant representations.

3.2 CMA-ES Controller Results

The CMA-ES controller was evaluated across three experimental configurations to thoroughly assess its capabilities and limitations.

Single round (random data): Mean reward of -555.32. The controller failed to learn any useful behavior, crashing every episode. CMA-ES with 50 generations and 64 candidates per generation could not find useful weights in the 1,156-dimensional parameter space.

Iterative data collection (3 rounds):

Round	Mean Reward	Improvement	Observation
1	-555.32	baseline	Random data, all crashes
2	-114.52	$4.8\times$ better	Controller data improved model quality
3	-143.25	regression	VAE posterior collapse (KL $\to$ 0.03)

Table 2: CMA-ES performance across iterative training rounds.

The $4.8\times$ improvement from Round 1 to Round 2 validates the iterative data collection hypothesis: when the controller collects data from states it actually visits, the world model learns better representations for those states.

Dream training: Training the controller inside the MDRNN's imagination with a learned reward predictor. The reward predictor achieved a loss of approximately 103, indicating it could not accurately estimate rewards from the mostly-crashing training data. Dream training produced controllers that performed well in imagination but poorly in reality (real eval: -578), revealing the gap between the world model's predictions and true environment dynamics.

3.3 PPO Controller Results

Replacing CMA-ES with PPO while keeping the identical World Models + ICM architecture produced dramatically better results. Table 3 shows the training progression.

PPO achieved in 200,000 timesteps ($\sim$1 minute on CPU) what CMA-ES could not achieve in 3 rounds of iterative training ($\sim$2 hours). The fundamental difference is information efficiency: CMA-ES receives one scalar reward per episode and cannot attribute outcomes to specific actions, while PPO computes per-timestep advantage estimates, extracting approximately $200\times$ more learning signal per episode.

Timesteps	Mean Reward	Agent Behavior
2,000	−92	Crashing on every episode
10,000	−59	Learning to fire engines, slowing descent
16,000	+9	First positive reward
80,000	+22	Hovering, attempting to approach landing pad
120,000	+74	Landing on some episodes
145,000	+99	Consistent landings
197,000	+145	Landing between the flags

Table 3: PPO training progression showing clear learning phases over 200,000 timesteps.

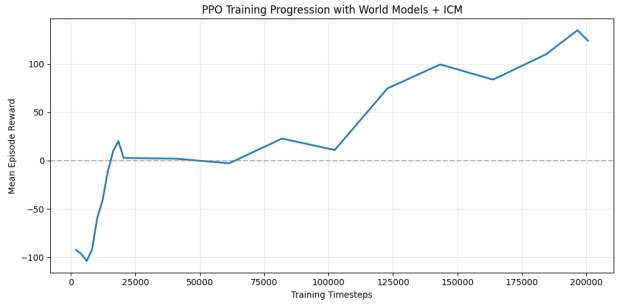

Figure 1: Mean episode reward during PPO training over 200,000 timesteps. The learning curve shows distinct phases: initial crashes, stabilization through engine control, and eventual successful landings between the flags.

Evaluation gap. However, evaluation performance showed a significant gap relative to training performance. Across multiple experimental runs:

Run Configuration	Mean Reward	Best Episode	Worst Episode
200k steps, 200 episodes data	−47.81	−7.46	−82.83
1M steps, 200 episodes data	−56.78	+76.78	−173.82
200k steps, loaded models	−225.66	−129.63	−336.88

Table 4: Evaluation results across different experimental configurations (all with $\eta = 0$).

This discrepancy between training rewards (+145) and evaluation rewards (−47 to −225) is attributable to **distribution shift**: the VAE and MDRNN were trained on random exploration data and produce less accurate encodings for the controlled trajectories that the trained policy generates. The agent encounters states during evaluation that the world model has never seen, degrading the quality of the 288D observation that PPO relies on.

3.4 Hyperparameter Sensitivity Analysis

Intrinsic reward scaling (η): This parameter controls the balance between task completion and exploration.

At $\eta = 0.01$, the cumulative intrinsic reward per episode ($\sim$100) competed directly with the landing

η	**Training Behavior**	**Issue**
0.01	Agent prefers hovering over landing	Curiosity dominates extrinsic reward
0.001	Agent attempts landing but inconsistent	Curiosity still competes with landing
0.0001	Agent learns to land (+145 training)	Sweet spot: curiosity aids early exploration
0.0	Agent learns but slower early progress	No exploration signal when rewards are sparse

Table 5: Effect of intrinsic reward scaling on agent behavior.

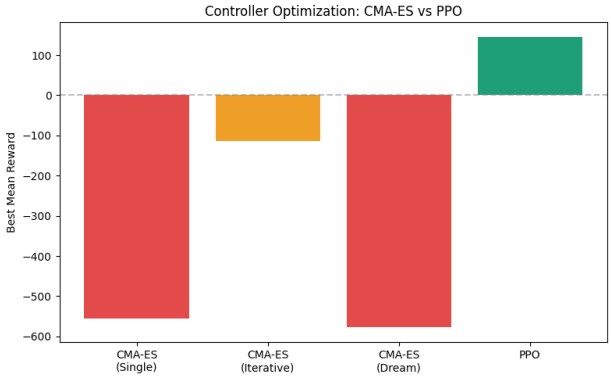

Figure 2: Best mean reward achieved by each controller optimization strategy

reward (+140), causing the agent to prefer exploring novel states (hovering) over completing the task. At $\eta = 0.0001$, the intrinsic reward per episode ($\sim$1) provided useful early exploration signal without interfering with task-oriented learning once the agent discovered landing.

VAE β parameter: β-warmup to 1.0 caused posterior collapse in iterative training rounds (KL divergence dropped to 0.03, meaning most latent dimensions encoded no information). Capping β at 0.5 prevented this while maintaining sufficient regularization.

3.5 CMA-ES vs PPO: Comparative Analysis

Metric	**CMA-ES**	**PPO**
Best training reward	-114	$+145$
Training time	$\sim$2 hours	$\sim$1 minute
Learning signal per episode	1 scalar	$\sim$200 per-timestep advantages
Uses ICM per-timestep?	No (sums to total)	Yes (per-timestep reward)
Gradient information?	None (black box)	Policy gradient

Table 6: Comparison of CMA-ES and PPO as controller optimization methods.

The results demonstrate that PPO is fundamentally better suited for this architecture because it can leverage both the per-timestep ICM reward signal and the rich 288D world model observation at the granularity they are designed for.

4 Conclusions

4.1 Summary

This project successfully implemented the complete World Models + ICM architecture from scratch and evaluated it on the LunarLander-v3 environment. The system comprises four custom neural network architectures totaling over 500,000 parameters: a VAE (108,488), a custom LSTM (300,032), a Mixture Density Network (83,525), and an ICM (13,060). All components—including the LSTM gates, MDN mixture of Gaussians with logsumexp loss, VAE with reparameterization trick and β-warmup, and ICM with detached target features—were implemented from scratch in PyTorch without pre-built modules.

The project explored three controller optimization strategies (CMA-ES, dream training, PPO), conducted iterative data collection experiments across multiple rounds, performed hyperparameter sensitivity analysis on both η and β, and documented the distribution shift challenge through systematic evaluation.

4.2 Strengths

The primary strength is the depth of exploration across the architecture. The transition from CMA-ES (-114 best) to PPO ($+145$ training) provided empirical evidence for the advantage of gradient-based optimization in this setting. The iterative data collection experiments ($4.8\times$ improvement from Round 1 to Round 2) validated a core hypothesis of the World Models approach: better data leads to better models leads to better policies. The η sensitivity analysis mapped the intrinsic-extrinsic reward tradeoff across four settings, providing practical guidance for balancing curiosity-driven exploration with task completion.

4.3 Limitations

The main limitation is distribution shift between training and evaluation. The world model components learned representations from random exploration data that degrade when processing the controlled trajectories of a trained policy. This explains the gap between training performance ($+145$) and evaluation performance (-47 to -225).

Additionally, LunarLander's low-dimensional state space (8D) and relatively dense reward structure mean that standard model-free PPO on raw observations solves the environment more efficiently than our World Models approach. The architecture's value lies not in this specific environment but in its scalability to domains where observations are high-dimensional (images), rewards are sparse, and environment interaction is expensive or dangerous.

4.4 Future Work

Several improvements could enhance performance. Iterative data collection with PPO would close the distribution shift gap by retraining all models on data from the improving policy. End-to-end training would align the latent representation with the policy's needs rather than optimizing for reconstruction. Discrete latent representations, as used in DreamerV2 and DreamerV3 (Hafner et al., 2020; 2023), would prevent posterior collapse entirely. Finally, applying this architecture to CarRacing-v2 with a convolutional VAE would demonstrate the visual compression capabilities that are the true strength of the World Models framework.

5 Works Cited

- Ha, D., & Schmidhuber, J. (2018). World Models. *arXiv preprint arXiv:1803.10122*.
- Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven Exploration by Self-supervised Prediction. *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, 2778–2787.
- Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to Control: Learning Behaviors by Latent Imagination. *International Conference on Learning Representations (ICLR 2020)*.
- Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2023). Mastering Diverse Domains through World Models. *arXiv preprint arXiv:2301.04104*.
- Hansen, N. (2016). The CMA Evolution Strategy: A Tutorial. *arXiv preprint arXiv:1604.00772*.
- Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*.
- Raffin, A., et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research, 22*(268), 1–8.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.
- Towers, M., et al. (2024). Gymnasium: A Standard Interface for Reinforcement Learning Environments. *arXiv preprint arXiv:2407.17032*.